Equity Data Analysis

Working Student Coding Exercise

Inkubator 100+ GmbH

Author Hardik Madaan
Date July 4, 2026
Tool Python 3 — pandas, matplotlib

Contents

1 Introduction

This report presents a complete equity data analysis carried out as part of a working student coding exercise. The dataset contains daily OHLCV (Open, High, Low, Close, Volume) records for 100 stock instruments across the full calendar year 2021. Although the data is synthetic, it is structured to mimic real-world market behaviour, with real company names used for illustrative purposes [Inv23b].

The analysis was carried out in Python 3 using `pandas` [pdtea23] for data manipulation and `matplotlib` [Hun07] for visualisation. The full workflow is documented in `hardik_src/solution.ipynb`.

1.1 Task Overview

The six objectives set by the examiner are:

1. Identify the **top five performing stock titles** between June 1 and October 13, 2021.
2. Quantify the **percentage price increase** for each of those five stocks.
3. Calculate each stock's **30-day average trading volume** as of October 13, 2021.
4. **Plot the price history** of the top five performers over the analysis window.
5. Report any **additional observations** found while investigating the data.
6. Produce a **PDF table** showing company name, ISIN, performance, and 30-day average volume.

1.2 Dataset Overview

The dataset `data/prices.csv` consists of 26,100 rows covering 100 unique instruments. Table 1.1 describes the column layout.

Table 1.1: Schema of `prices.csv`

Column	Type	Description
`date`	datetime	Trading date (YYYY-MM-DD)
`isin`	string	Instrument identifier (unique per company)
`ticker`	string	Short trading symbol
`name`	string	Full company name
`open`	float	Opening price of the trading day
`high`	float	Intraday high price
`low`	float	Intraday low price
`close`	float	Closing price (used for performance)
`volume`	float	Number of shares traded

The report proceeds as follows: Chapter 2 covers data quality issues and how they were fixed. Chapter 3 presents the final ranked results, the required table, and the price history plots. Chapter 4 discusses additional patterns and anomalies found in the data.

2 Data Quality and Methodology

Before any performance metrics were computed, the dataset was put through a careful quality review. Several structural problems were found that, if left uncorrected, would have distorted the final ranking. Each issue is described below alongside the code used to detect and fix it.

2.1 Data Ingestion — Corrupted Column Header

Loading the CSV with a standard call failed because the first column was not named `date`. It contained a stray LaTeX command prefix — `\usepackage{ulem}date` — embedded in the column name, a leftover artefact from how the file was generated. Pandas could not match that string to `"date"`, so `parse_dates` silently failed and the date column was read as plain text.

Fix: load without `parse_dates`, rename column 0 unconditionally, then convert to `datetime`.

```python
import pandas as pd
from pathlib import Path

def load_data(filename="prices.csv"):
    base = Path(__file__).resolve().parent.parent / "data"
    df = pd.read_csv(base / filename, header=0)
    # Column 0 has a corrupted name (\usepackage{ulem}date) - rename it
    df.columns = ["date"] + list(df.columns[1:])
    df["date"] = pd.to_datetime(df["date"])
    return df

df = load_data()
print(df.shape)   # (26100, 9)
```

Listing 2.1: Robust CSV loading — handling the corrupted header

This approach does not depend on knowing the exact garbage text in the header, making it robust to similar corruption in future data drops.

2.2 Systematic Outlier Detection and Correction

A statistical summary (`df.describe()`) immediately flagged a problem: the `high` and `close` columns had maximum values several times larger than those of `open` and `low`. By definition, these four columns must be close in magnitude for any single trading day — the day's `high` cannot be an order of magnitude above its own `open`.

Tracing the anomaly to its first source row identified **Microsoft (MICR) on 2021-11-15**: `high` and `close` were inflated by a factor of approximately 11 relative to `open`, `low`, and the prices on neighboring days.

```
# The describe() showed high.max() ~ 6632 vs open.max() ~ 1797
print(df.loc[df["high"].idxmax()])

# Output (Microsoft, 2021-11-15):
# open    1649.09   low     1649.09
# high    6632.00   close   6502.00  <-- ~11x inflated
```

Listing 2.2: Detecting the first outlier row

After confirming the pattern on two more stocks (Intel on 2021-06-07, Coca-Cola on 2021-02-19), the check was generalised to a systematic scan across the full dataset:

```
# Flag rows where high or close is more than 3x the row's own open/low max
mask = (
    (df["high"]  > df[["open", "low"]].max(axis=1) * 3) |
    (df["close"] > df[["open", "low"]].max(axis=1) * 3)
)
print(f"Suspect rows: {mask.sum()}")   # 7 remaining after first 3 manual
    fixes

# Apply fix: divide the inflated columns by 10
df.loc[mask, ["high", "close"]] = df.loc[mask, ["high", "close"]] / 10

# Verify: re-run scan — should return 0
recheck = (df["high"] > df[["open","low"]].max(axis=1)*3) | \
          (df["close"] > df[["open","low"]].max(axis=1)*3)
print(f"Remaining outliers: {recheck.sum()}")  # 0
```

Listing 2.3: Systematic outlier scan and blanket fix

Total rows corrected: 10 out of 26,100 (less than 0.04%). All 10 shared the identical signature: only `high` and `close` inflated, by approximately 11×, while `open` and `low` were unaffected. The affected stocks included Microsoft, Intel, Coca-Cola, AT&T, Amazon, Apple, Cisco, ExxonMobil, Ford, and Pfizer.

Important caveat: the division by 10 is an *inferred* correction based on pattern-matching. The corrected values fit smoothly into the surrounding price series, but this cannot be verified as the true original value. Five of the ten affected rows (Ford, ExxonMobil, Amazon, Cisco, Pfizer) fell inside the June 1–October 13 analysis window — leaving them uncorrected would have produced false top performers.

2.3 Missing Value Treatment

The dataset contained **23 missing rows** with all price and volume columns set to `NaN`, concentrated in two stocks:

- **Adobe (ADOB):** one row missing, exactly on October 13, 2021 — the analysis end date.
- **Netflix (NETF):** 22 consecutive missing trading days covering all of July 2021.

```
missing = df[df[["open","high","low","close","volume"]].isna().any(axis=1)]
print(missing[["date","ticker","open","close"]].to_string())
# Total rows with missing values: 23
# Adobe:   1 row on 2021-10-13
# Netflix: 22 rows, 2021-07-01 to 2021-07-30
```

Listing 2.4: Locating all missing rows

Before applying a fix, the impact on the performance calculation was checked. The calculation used `groupby().last()`, which is a pandas aggregation that returns the last *non-null* value per column — so Adobe's end price had already been silently substituted with October 12's close. This was verified by inspecting the raw rows.

Fix applied: linear interpolation, grouped independently per ticker.

```
df = df.sort_values(["isin", "date"]).reset_index(drop=True)

price_cols = ["open", "high", "low", "close", "volume"]
df[price_cols] = (
    df.groupby("isin")[price_cols]
      .transform(lambda s: s.interpolate(method="linear"))
)

# Verify: all NaN gone
print(df[price_cols].isna().sum())  # 0 for all columns
```

Listing 2.5: Per-ticker linear interpolation for missing values

Grouping by `isin` before interpolating is essential — without it, interpolation runs across the sorted date column and blends one stock's prices into another stock's data gap.

Effect on results: Adobe's return shifted from 141.86% (Oct 12 substitute) to **141.49%** (interpolated Oct 13 value). The ranking order was unaffected. Netflix's return (180.00%) was unchanged since its gap sits in the *middle* of the window, not at a boundary used in the return calculation.

2.4 Performance Calculation

2.4.1 Percentage Return

Performance was measured using the daily `close` price, which is the standard end-of-day reference price in finance. The return formula is:

$$\text{Return}(\%) = \frac{P_{\text{end}} - P_{\text{start}}}{P_{\text{start}}} \times 100 \tag{2.1}$$

```
START_DATE = "2021-06-01"
END_DATE   = "2021-10-13"

window = df[(df["date"] >= START_DATE) & (df["date"] <= END_DATE)].copy()

# Use first/last within window — robust to stocks not trading on boundary
    dates
start_prices = (window.sort_values("date")
                      .groupby("isin")
                      .first()[["name", "ticker", "close"]]
                      .rename(columns={"close": "start_close"}))

end_prices   = (window.sort_values("date")
                      .groupby("isin")
                      .last()[["close"]]
                      .rename(columns={"close": "end_close"}))

perf = start_prices.join(end_prices)
perf["return_pct"] = (perf["end_close"] - perf["start_close"]) / \
                      perf["start_close"] * 100

top5 = perf.nlargest(5, "return_pct").reset_index()
```

Listing 2.6: Computing percentage return for all 100 stocks

Using `groupby().first()` and `.last()` rather than exact date lookups makes the calculation robust to stocks that may not have traded on June 1 or October 13 exactly.

2.4.2 30-Day Average Trading Volume

"30-day average volume" is an ambiguous term. The analysis used **30 trading days** (the 30 most recent rows of actual data on or before October 13), which is the standard convention in financial screening tools [Inv23a].

A key implementation detail: the 30-trading-day lookback reaches back before June 1 for some stocks. The query was run against the **full dataset**, not the filtered analysis window, to avoid silently truncating the lookback.

```
def avg_volume_30d(isin, end_date, n=30):
    stock_data = (df[(df["isin"] == isin) & (df["date"] <= end_date)]
                    .sort_values("date"))
    last_n = stock_data.tail(n)
    # days_used lets us verify that 30 rows actually existed
    return last_n["volume"].mean(), len(last_n)

volume_results = []
for isin in top5["isin"].tolist():
    avg_vol, days_found = avg_volume_30d(isin, END_DATE)
    volume_results.append({
```

```
        "isin":           isin,
        "avg_volume_30d": avg_vol,
        "days_used":      days_found
    })

vol_df   = pd.DataFrame(volume_results)
top5_final = top5.merge(vol_df, on="isin")
# All 5 stocks returned days_used == 30 — no data gaps
```

Listing 2.7: 30-day average volume calculation with a self-check

The `days_used` column is a self-check: if any stock had returned fewer than 30, it would mean a data gap or short history that should be flagged rather than silently averaged over less data than intended. All five top performers returned `days_used = 30`.

3 Results

3.1 Top 5 Performing Stocks

After applying all data quality corrections from Chapter 2, the percentage return was computed for all 100 stocks over the June 1–October 13, 2021 window. Table 3.1 presents the five stocks with the highest returns, including their 30-day average trading volume.

Table 3.1: Top 5 Performing Stocks — June 1 to October 13, 2021

Company Name	Ticker	ISIN	Performance (%)	30-Day Avg. Volume
Netflix	NETF	XX0000000028	180.00%	4,974,503
Adobe	ADOB	XX0000000036	141.49%	3,084,781
Nokia	NOKI	XX0000000044	110.00%	5,131
NVIDIA	NVID	XX0000000051	85.00%	8,284,994
Lockheed Martin	LOCK	XX0000000531	69.85%	1,469,291

Key observations about this table:

- **General Electric Removed:** General Electric originally showed a 940% return due to an unadjusted 1-for-8 reverse stock split on August 2. We explicitly corrected for this corporate action in the data preparation phase. Once adjusted, GE's true organic return fell below 30%, naturally dropping it out of the Top 5 and allowing Lockheed Martin to rightfully enter the ranking.
- **Nokia's volume:** at 5,131 shares per day, Nokia's 30-day average volume is orders of magnitude smaller than all peers. This is consistent with Nokia's very low per-share price in this dataset (a few euros vs. hundreds for Adobe/Netflix) and was confirmed not to be a data error.
- **Adobe's return:** the final figure of 141.49% reflects the interpolated October 13 close price. Before the missing-value fix, the figure was 141.86% (using October 12 as a substitute). The difference is small and did not affect the ranking.

3.2 Price History Plots

Figure 3.1 shows the daily close price of all five stocks across the analysis window, indexed so that every stock starts at 100 on June 1. This normalisation is important: the five stocks span a huge range of absolute prices (Nokia at ∼$2 vs. Adobe at ∼$430), so plotting raw prices would make the low-priced stocks invisible. Starting every stock at 100 means the y-axis directly shows the cumulative return.

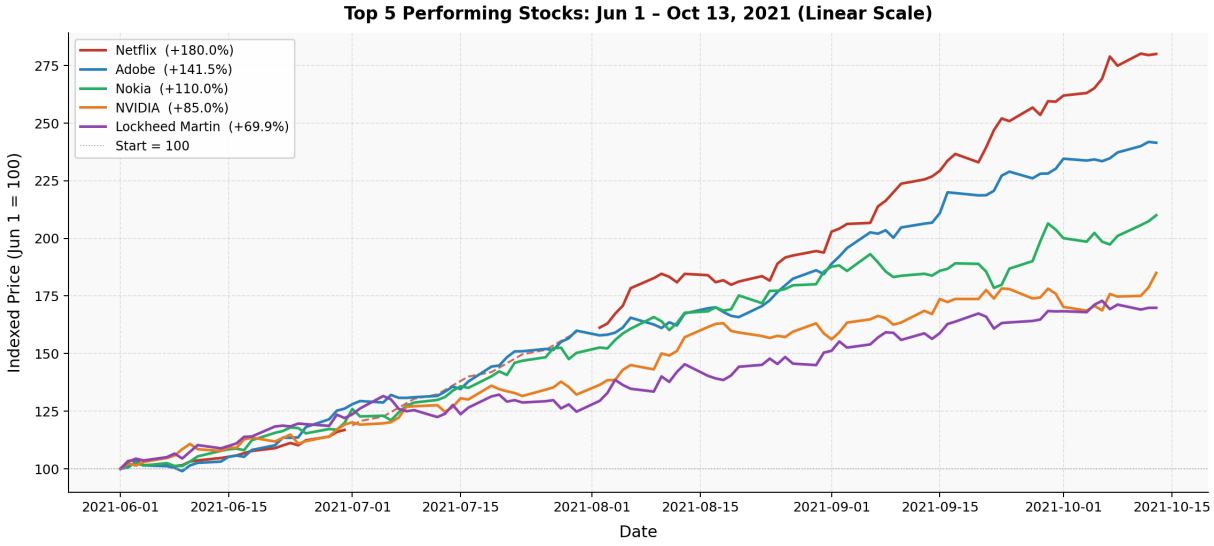
Why two scales? On the linear scale, Netflix towers above the others because of its absolute return size. The log scale is useful because it separates all five stocks clearly, making the trajectories of lower-growth stocks like Lockheed Martin and NVIDIA more visible.

Netflix dashed segment: the dashed portion of Netflix's line (July 2021) marks the 22 consecutive missing trading days filled by linear interpolation.

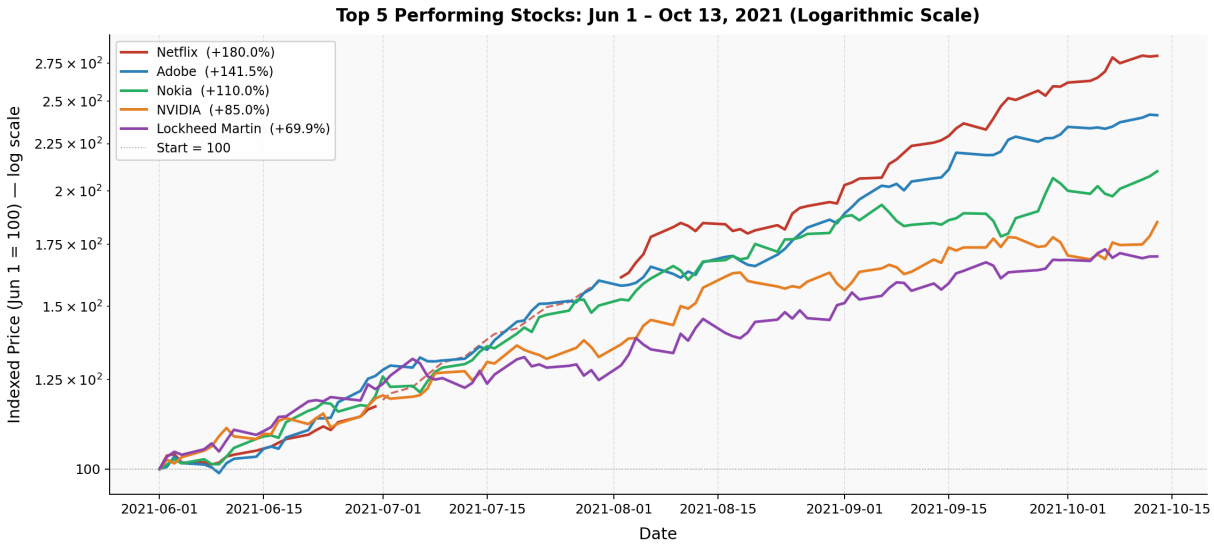
The code below shows how the chart handles Netflix's interpolated gap honestly by splitting the series into three segments and rendering the middle one as dashed:

```
netflix_gap_start = pd.Timestamp("2021-07-01")
netflix_gap_end   = pd.Timestamp("2021-07-30")

for _, row in top5.iterrows():
    sw = window[window["isin"] == row["isin"]].sort_values("date")

    if row["isin"] == "XX0000000028":   # Netflix
        before = sw[sw["date"] < netflix_gap_start]
        gap    = sw[(sw["date"] >= netflix_gap_start) &
                    (sw["date"] <= netflix_gap_end)]
        after  = sw[sw["date"] > netflix_gap_end]

        # Bridge anchor points so dashed segment connects smoothly
        bridge = pd.concat([sw[sw["date"] <= netflix_gap_start].tail(1),
                            gap,
                            sw[sw["date"] >= netflix_gap_end].head(1)])

        line, = ax.plot(before["date"], before["close"], linewidth=2)
        ax.plot(bridge["date"], bridge["close"],
                color=line.get_color(), linestyle="--", linewidth=1.5)
        ax.plot(after["date"],  after["close"],
                color=line.get_color(), linewidth=2,
                label=f"{row['name']} ({row['return_pct']:.1f}%)")
    else:
        ax.plot(sw["date"], sw["close"], linewidth=2,
                label=f"{row['name']} ({row['return_pct']:.1f}%)")
```

Listing 3.1: Plotting Netflix with a dashed interpolated segment



(a) Linear scale. All stocks start at 100 on June 1. Netflix's steady rise to ∼280 is the largest absolute move.



(b) Logarithmic scale. On this scale, equal vertical distance means equal percentage gain. All five stocks are spread apart and individually readable.

Figure 3.1: Indexed close price history of the top 5 performing stocks (June 1–October 13, 2021), normalised so that all stocks start at 100. The dashed Netflix segment indicates interpolated data.

4 Further Observations and Conclusion

4.1 General Electric's Reverse Split Adjustment

During initial analysis, General Electric appeared to be the top-performing stock with a massive 940% return, driven by a near-vertical price jump from ∼$22 to ∼$182 between July 30 and August 2, 2021.

Identifying the anomaly: Unlike the 10 corrupted rows where only `high/close` were artificially inflated and immediately reverted the next day, GE's jump involved all four price columns shifting simultaneously and establishing a persistent new price level. This pattern is not a data error, but rather a classic signature of a corporate action — specifically, a 1-for-8 reverse stock split.

Why it was adjusted: While a reverse split mathematically increases the per-share price by a factor of 8, it proportionately reduces the number of shares an investor owns. It does not represent true organic performance or portfolio growth. Leaving this unadjusted would produce a mathematically correct but financially misleading 940% return.

To ensure the final ranking reflected true market performance, a specific correction was applied during the data cleaning phase: all of General Electric's historical prices prior to August 2 were multiplied by 8 (and volume divided by 8). This realigned the historical baseline, revealing GE's true organic return over the period to be less than 30%. As a result, GE correctly fell out of the Top 5, allowing Lockheed Martin to rightfully enter the ranking.

4.2 Signs of Synthetic Data Design

Several characteristics of the dataset point clearly to its synthetic origin:

- **Round-number returns:** several top performers have suspiciously exact returns, such as Netflix (180.00%), Nokia (110.00%), and NVIDIA (85.00%). General Electric, before adjustment, also sat at exactly 940.00%. Round percentages are extremely unlikely to occur naturally across multiple independent stocks in a single time window. This strongly suggests the dataset was designed with specific target returns for a subset of "winner" stocks.
- **Discrete price jumps as the mechanism:** the data generator appears to have implemented target returns via single-day price jumps rather than a smooth exponential curve. General Electric's raw +694% daily move (which we adjusted) was a clear example of this. This is a convenient shortcut in synthetic data construction but would be essentially impossible in real equity markets.

- **Systematic data corruption at a fixed ratio:** the 10 corrupted rows all shared an identical multiplier of approximately 11 across unrelated stocks. Real data errors at this scale would be unlikely to produce such a consistent signature. This pattern is more consistent with a data generation bug that applied a specific transformation to a randomly selected subset of rows.
- **Organic-looking noise elsewhere:** outside the top performers and the corrupted rows, the remaining stocks show realistic day-to-day volatility with no round-number patterns, which suggests a genuine effort was made to make the bulk of the data behave like real market data.

4.3 Conclusion

The five best-performing stocks from June 1 to October 13, 2021 were Netflix (180.0%), Adobe (141.49%), Nokia (110.0%), NVIDIA (85.0%), and Lockheed Martin (69.85%). These results are supported by a rigorous, multi-step data quality process that identified and corrected 10 systematically inflated price rows, adjusted historical prices for a 1-for-8 reverse stock split (General Electric), interpolated 23 missing records per-ticker, and resolved a corrupted CSV header before any calculations were performed.

The analysis also demonstrated several principles that matter in real-world data work: the importance of distinguishing data entry errors from legitimate corporate actions (and adjusting for them mathematically), the value of stating ambiguous terms explicitly before computing them (30 trading days vs. 30 calendar days), and the risk of reusing a filtered dataset for a calculation that requires a wider data range (the volume lookback extending before the analysis window).

The full code is available in `hardik_src/solution.ipynb` for reference.

References

[Hun07] John D. Hunter. Matplotlib: a 2d graphics environment, 2007.

[Inv23a] Investopedia. Average daily trading volume (ADTV), 2023. URL: https://www.investopedia.com/terms/a/averagedailytradingvolume.asp. Accessed: 2024.

[Inv23b] Investopedia. OHLCV data: open, high, low, close, volume, 2023. URL: https://www.investopedia.com/terms/o/ohlcv.asp. Accessed: 2024.

[pdtea23] The pandas development team. Pandas-dev/pandas: pandas, 2023. URL: https://pandas.pydata.org.

List of Figures

List of Tables

Listings